

Большие модели на одной GPU

Введение в Эффективные Системы Глубокого Обучения

Феоктистов Дмитрий Дмитриевич, Yandex Research, BRAIn Lab {@trandelik}

Лекция 08, осень 2026

Модель помещается. Обучение – нет.

Хотим дообучить модель на 7B параметров.

BF16-веса
14 GB



Одна GPU
24 GiB

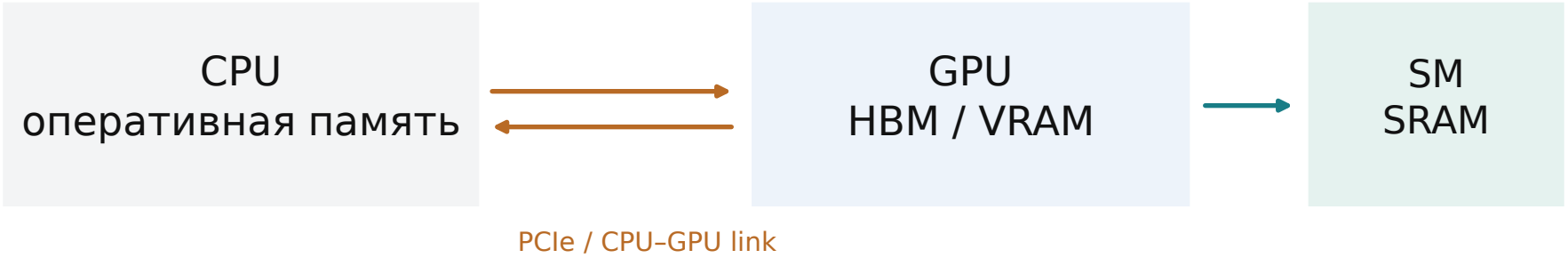
А потом вызываем backward() и optimizer.step()…

Что ещё должно поместиться?

Что будем менять?

- **Расписание:** microbatch, accumulation, checkpointing.
- **Размещение:** GPU ↔ CPU, prefetch и overlap.
- **Обучаемые параметры:** LoRA и QLoRA.
- **Обучающий сигнал:** ZO, MeZO и опыт Noise.

Ограничение – не только объём VRAM



- **Capacity:** хватит ли места для одновременно живых данных?
- **Bandwidth:** успеем ли доставить данные к моменту использования?
- **Compute:** сколько работы добавим ради экономии памяти?

Что занимает память?

$$M_{\text{tensors}}^{\text{peak}} = \max_t [M_W(t) + M_G(t) + M_O(t) + M_A(t) + M_T(t)]$$

Обозначение	Что это
W / G / O	Веса (и master copy) / градиенты / states
A	Сохранённые для backward активации
T	Временные буферы вычислений и optimizer step

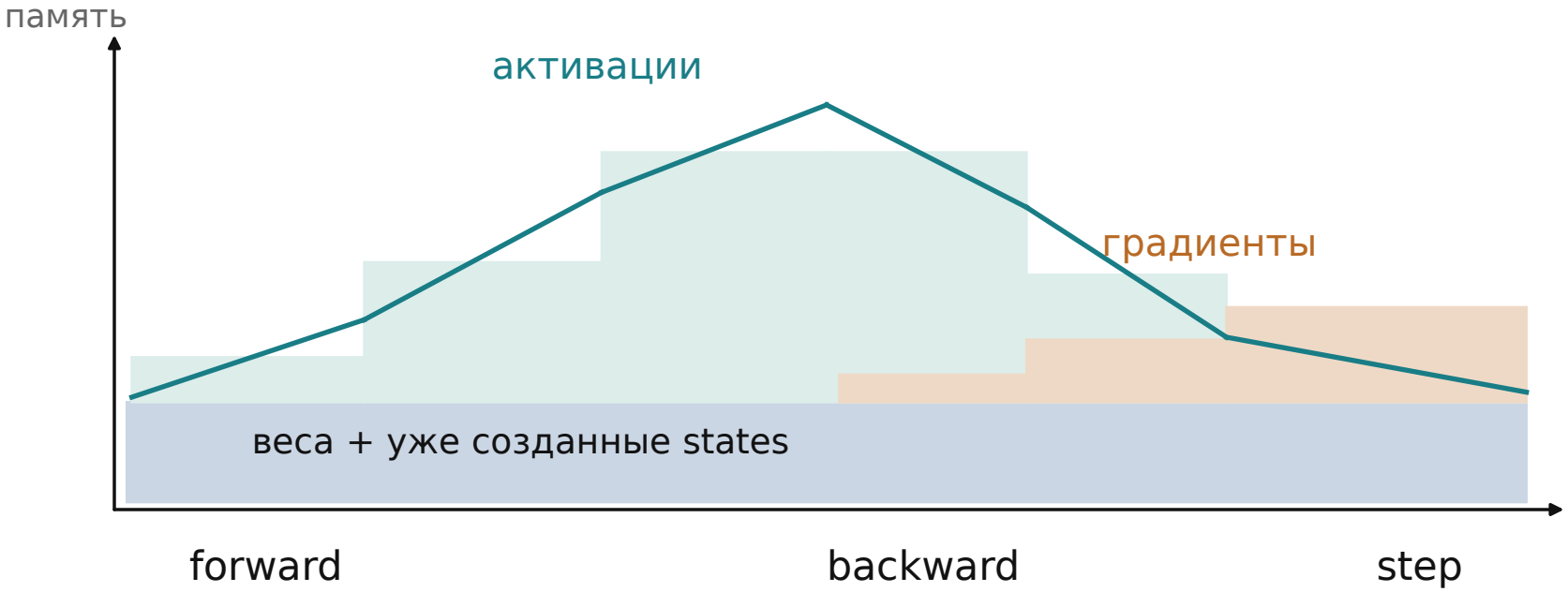
Плюс запас на CUDA runtime, allocator и внешние библиотеки.

7B: цена состояния обучения

Объект	Пример хранения	Память
Веса	7B × 2 B, BF16	14 GB ≈ 13,0 GiB
Градиенты	7B × 2 B, BF16	14 GB ≈ 13,0 GiB
Два момента Adam	7B × 8 B, FP32	56 GB ≈ 52,2 GiB
Итого в этом примере	Без master weights и активаций	84 GB ≈ 78,2 GiB

Загрузка BF16-весов ещё не означает, что поместится обучение.

Память меняется в течение шага



Где именно возникает OOM?

Момент	На что смотреть в первую очередь
Загрузка	Веса, копии параметров, staging-буферы
Forward	Активации, logits, длина последовательности
Backward	Сохранённые тензоры + появляющиеся градиенты
Optimizer step	Ленивые states и временные буферы

Один и тот же OOM – разные решения

Что доминирует	Что пробовать
Активации	Microbatch → checkpointing → activation offload
Градиенты и states	PEFT, другой optimizer, optimizer offload
Базовые веса	Квантизация, parameter offload
Временные буферы	Формы входов, fused/chunked kernels

Первый шаг: уменьшим microbatch, сохранив нужный effective batch.

Сначала отделим batch от microbatch

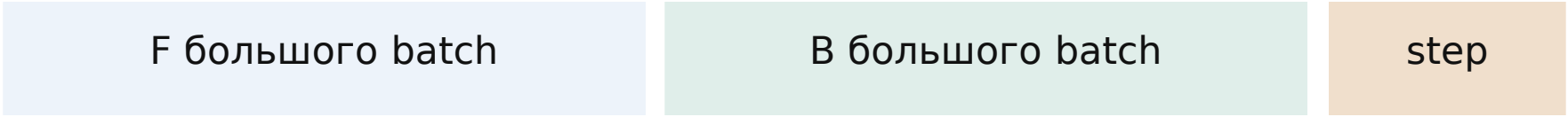
На одной GPU

$$B_{\text{effective}} = B_{\text{micro}} \cdot K$$

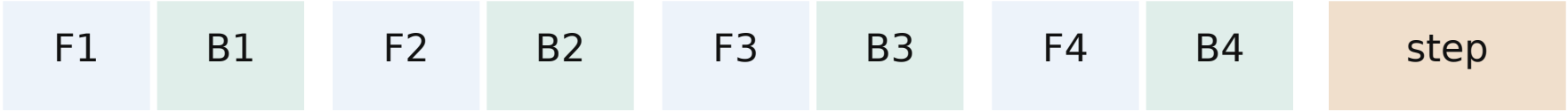
- Microbatch выбираем по памяти **и производительности**.
- K определяет, сколько backward выполняем до одного step.
- Большая карта позволяет увеличить microbatch; accumulation доводит effective batch до выбранного значения.

Gradient accumulation: тот же момент обновления

Без накопления



С накоплением: например, K = 4



Что мы выиграли и чем заплатили?

- Одновременно живут активации **одного microbatch**.
- Память весов, накопленных градиентов и states почти не меняется.
- Маленькие matmul могут хуже загружать GPU; растёт число запусков.
- Accumulation управляет effective batch, но сам по себе не ускоряет обучение.

Сохраняем смысл шага оптимизации

- **Нормируем loss:** по примерам или по валидным токенам.
- **Clip и step:** после полного окна накопления.
- Scheduler согласован с обновлениями или числом обработанных токенов.
- При том же effective batch новый LR не следует из изменения K.

Microbatch уже равен 1. Что дальше?

А что, если даже на одном примере не удаётся закончить forward?

- Accumulation уменьшает batch, но не глубину графа.
- Сохранённые активации всё ещё накапливаются от слоя к слою.
- **Попробуем не хранить всё до backward.**

Что именно нужно backward?

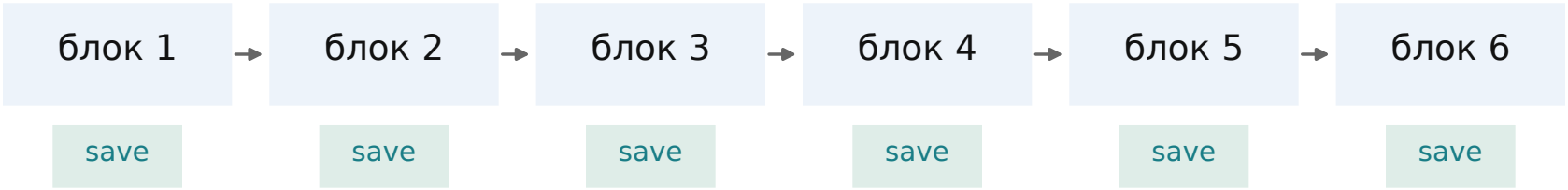
$$Y = WX, \qquad G = \frac{\partial L}{\partial Y}$$

$$\nabla_W L = G X^\top, \qquad \nabla_X L = W^\top G$$

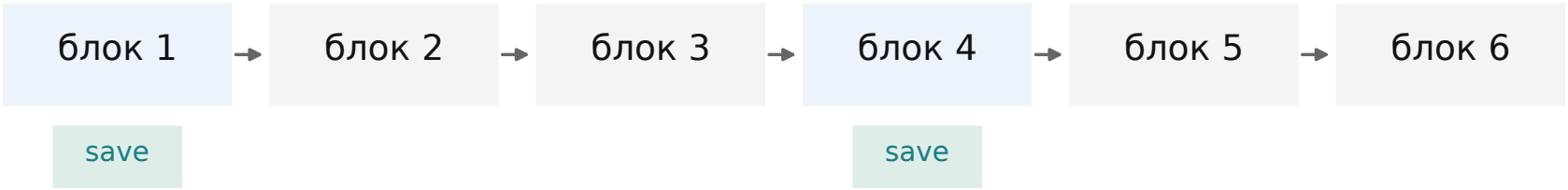
- Для градиента по весам нужен **вход X**.
- Для передачи градиента дальше нужны **веса W**.
- Сохранённые тензоры живут до своего использования в backward.

Можно не хранить всё до backward

Обычный forward

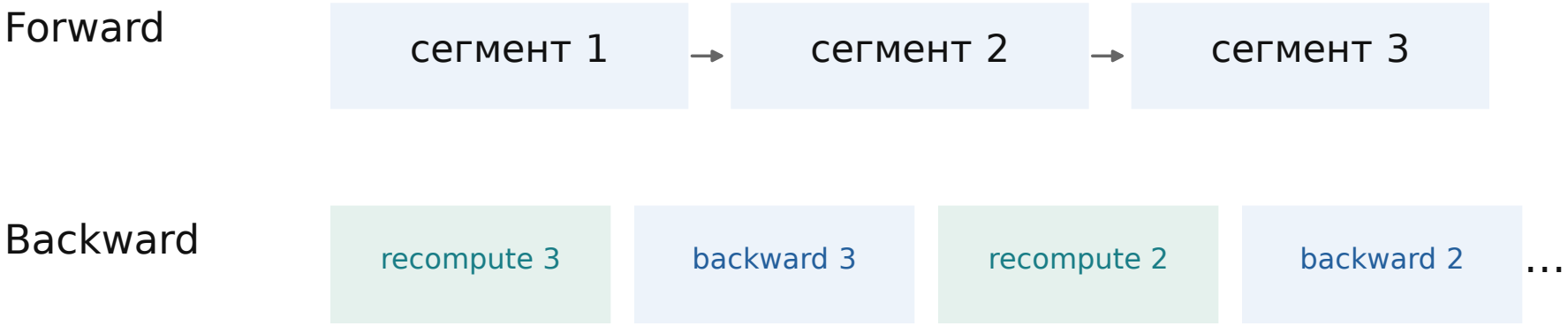


Forward с checkpointing



Храним границы регионов. Внутренние значения восстановим позже.

Checkpointing: пересчёт в нужный момент



- Восстановили локальные значения → использовали → освободили.
- Не нужно восстанавливать весь forward одновременно.

Где ставить чекпоинты?

L одинаковых блоков. В каждом сегменте k блоков.



Границы: примерно L/k

Восстановление сегмента: k

$$M_A(k) \approx a\left(\frac{L}{k} + k\right)$$

Считаем активации. Веса, градиенты и states здесь не учтены.

Chen et al.: sublinear memory, 2016

Баланс: примерно √L чекпоинтов

Минимизируем память границ и одного восстановленного сегмента:

$$\frac{L}{k} + k \geq 2\sqrt{L}, \qquad k^* \approx \sqrt{L}$$

$$N_{\text{checkpoints}} \approx \frac{L}{k^*} \approx \sqrt{L}, \quad M_A \approx 2a\sqrt{L}$$

Чекпоинт примерно через каждые √L слоёв.

Для одинаковых блоков и одного уровня пересчёта. В общем графе нужен профиль.

Chen et al.: square-root strategy

Checkpointing в PyTorch

Внутри forward оборачиваем выбранные блоки:

```
from torch.utils.checkpoint import checkpoint

for block in self.blocks:
    hidden = checkpoint(
        block, hidden, use_reentrant=False
    )
```

loss.backward() и optimizer.step() вызываются как обычно.

Hugging Face: model.gradient_checkpointing_enable(...)

Повторный forward должен быть тем же вычислением

- **Dropout:** согласованное RNG-состояние важно для того же градиента.
- Повторные записи в буферы, счётчики и другие side effects требуют внимания.
- Перенос на новое устройство внутри региона осложняет восстановление RNG.
- Non-reentrant вариант поддерживает больше сценариев autograd.

А зачем пересчитывать дорогой matmul?

Один checkpoint-регион



- Дорогие операции – кандидаты на сохранение результата.
- Дешёвые операции – кандидаты на пересчёт.
- Политика зависит от **размера, lifetime и стоимости восстановления**.

Selective checkpointing: политика внутри региона

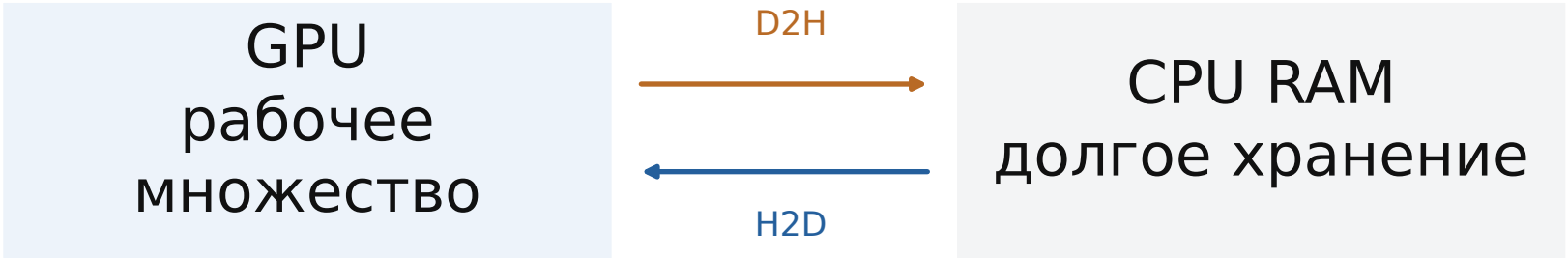
- Выбираем, **результаты каких операций** сохранять при forward.
- Остальные значения получаем повторным вычислением при backward.
- В PyTorch: policy и CheckpointPolicy для выбранных ATen-операций.
- torch.compile тоже может выбирать пересчёт; его поведение нужно профилировать.

Уже знакомый пример – FlashAttention

- Из предыдущей лекции: attention matrix не материализуется целиком.
- Часть значений восстанавливается во время backward.
- **Экономия памяти не всегда означает замедление.**
- Меньше HBM I/O может окупить пересчёт и дать **ускорение**.
- Поэтому эффект checkpointing измеряем поверх уже эффективного attention.

FlashAttention, Dao et al., 2022

А обязательно хранить тензор на GPU?



Значение нужно позже – держим его ближе к месту хранения, а к моменту вычисления возвращаем на GPU.

Если есть checkpointing, зачем offloading?

Пересчёт уменьшил сохранённые активации. Что осталось?

Ограничение checkpointing	Что даёт offloading
Веса и optimizer states	Могут храниться на CPU: checkpointing их не убирает.
Границы checkpoint-регионов	Можно вынести границы; внутренности пересчитывать.
Слишком дорогой пересчёт	Перенос иногда дешевле повторного compute.

Комбинируем методы, когда они экономят разные статьи памяти.

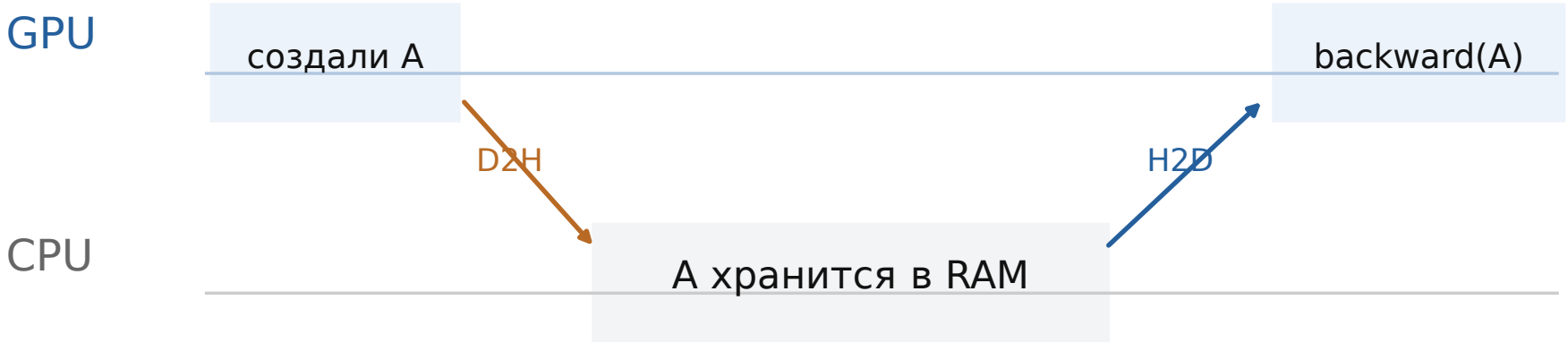
PyTorch: recomputation and saved tensors

Что именно можно offload?

Объект	Когда нужен на GPU	Чем платим
Активации	При backward соответствующего региона	Копиями и RAM
Параметры	При forward и backward слоя	Повторными загрузками
Optimizer states	Зависит от места выполнения step	CPU compute / передачами

ZeRO-Offload, Ren et al., 2021

Activation offload: до следующего использования



GPU-копию можно освободить, если её больше никто не удерживает.

Передачи не бесплатны

$$t_{\text{copy}} \gtrsim t_{\text{latency}} + \frac{S}{BW_{\text{effective}}}$$

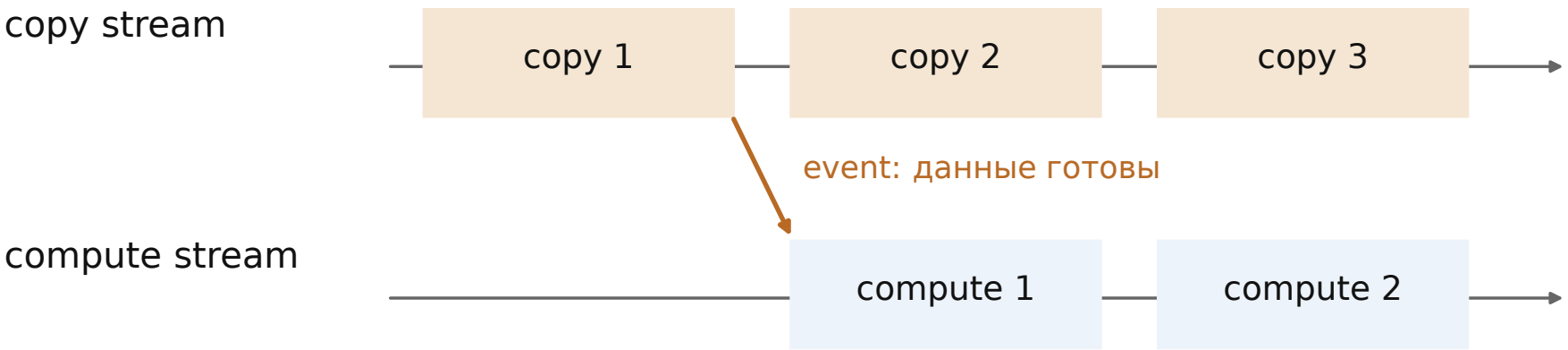
Иллюстрация при эффективной скорости 16 GiB/s:

1 GiB → CPU
≥ 62,5 ms

1 GiB → GPU
≥ 62,5 ms

Можно ли не ждать эти копирования?

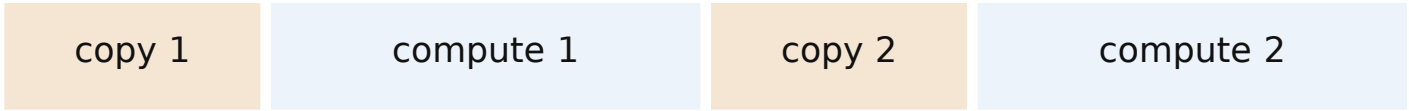
Чтобы GPU не ждала копирования



Порядок внутри stream есть. Зависимости между streams задаём явно.

Prefetch: переносим до момента использования

Последовательно



С prefetch

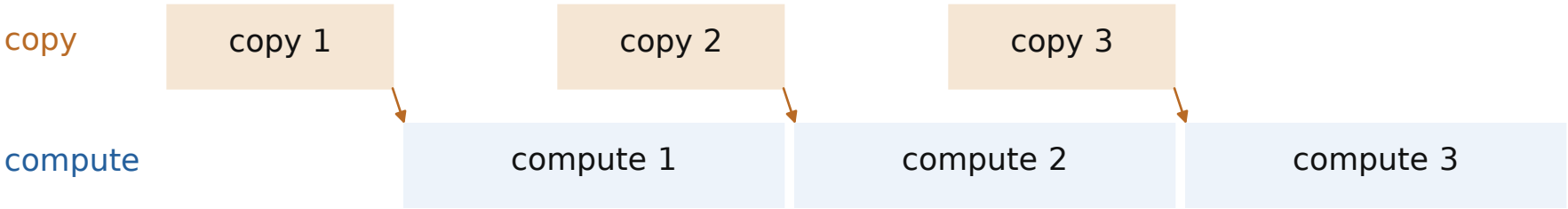


Схема. В установившемся режиме ориентируемся на max(compute, copy).

Что нужно для корректного overlap?

- **Pinned host memory** и поддержка асинхронных передач.
- Копия и вычисление в разных streams; между зависимыми операциями – event.
- Исходный буфер нельзя менять до завершения копирования.
- Lifetime GPU-буфера: allocator должен знать об использовании в другом stream.

CUDA Events: время и зависимости

- **Event с timing:** измеряет интервал на GPU.
- **record + wait_event:** связывает producer и consumer streams.
- CPU может продолжать работу; ожидание ставится в нужный stream.
- Для offloading важен и полный wall-clock шага с завершением всей работы.

Prefetch тоже занимает память

Данные, которые должны находиться на GPU одновременно:

Текущий блок вычисляется	Следующий блок уже загружен	Временные буферы / outputs

- Чем раньше prefetch, тем длиннее residency.
- Double buffering ускоряет pipeline, но требует запаса VRAM.

Хранить, пересчитывать или переносить?

$$\min_{\pi} T_{\text{step}}(\pi) \quad \text{s. t.} \quad M_{\text{peak}}(\pi) \leq M_{\text{budget}}$$

Сохранить	Пересчитать	Перенести
VRAM и lifetime	Compute и зависимости	Link, RAM и prefetch

Решение зависит от графа и расписания, а не только от FLOPs операции.

π – политика сохранения, пересчёта и размещения тензоров.

Проверяем выигрыш на всём training step

- Сначала warmup и создание optimizer states; отдельно смотрим первый шаг.
- Измеряем **peak allocated**, reserved и потребление CPU RAM.
- Смотрим timeline: где передачи, ожидания и фактический overlap?
- Сравниваем время при одинаковых входах, microbatch и effective batch.

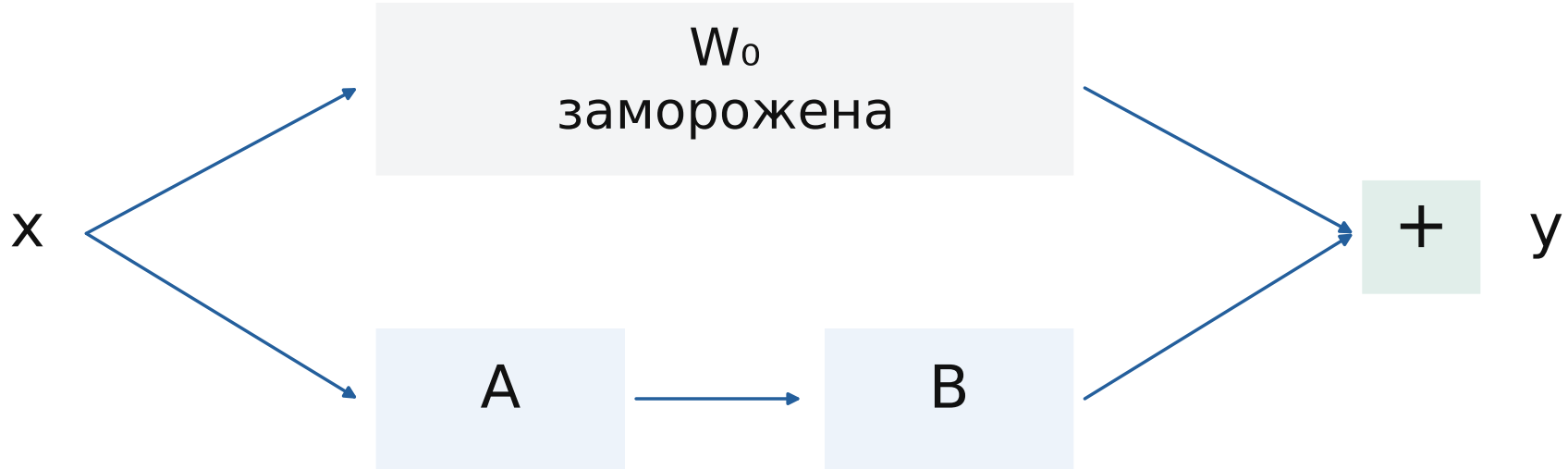
А нужно обучать все параметры?

Пересчёт и перенос меняют исполнение. Число обучаемых параметров то же.

- **PEFT:** обучаем небольшую часть параметров или добавленные модули.
- Bottleneck adapters: небольшая обучаемая ветка в слоях.
- Prompt / prefix tuning: обучаемые представления в контексте или attention.
- **LoRA:** низкоранговое приращение к выбранным матрицам.

LoRA, Hu et al., 2021

LoRA: обучаем приращение низкого ранга



обучаем

$$y = W_0 x + \frac{\alpha}{r} B(Ax)$$

LoRA: Low-Rank Adaptation of Large Language Models

LoRA: масштаб и инициализация

$$W = W_0 + \frac{\alpha}{r} BA, \qquad A \in \mathbb{R}^{r \times d_{in}}, \quad B \in \mathbb{R}^{d_{out} \times r}$$

- Типичная инициализация: A случайная, **B = 0**.
- В начале функция совпадает с замороженной базой.
- Первый шаг обновляет B; у A градиент в этот момент нулевой.
- Две нулевые матрицы остановят обучение этой ветки.

LoRA: parameterization and initialization

LoRA: градиенты обучаемых адаптеров

$$Z = AX, \qquad Y = W_0 X + \gamma BZ, \qquad G = \frac{\partial L}{\partial Y}$$

$$\nabla_B L = \gamma G Z^\top = \gamma G X^\top A^\top$$

$$\nabla_A L = \gamma B^\top G X^\top$$

$$A \leftarrow A - \eta \nabla_A L, \qquad B \leftarrow B - \eta \nabla_B L$$

γ = α/r. Показан шаг SGD; AdamW использует те же градиенты.

LoRA: chain rule для низкоранговой ветки

Заморозили веса. Почему backward остался?

$$Y = W_0 X + \gamma BAX, \qquad G = \frac{\partial L}{\partial Y}$$

$$\nabla_X L = (W_0 + \gamma BA)^\top G$$

- Не считаем и не храним градиент **по W_0**.
- Но передаём градиент **по X** к предыдущим adapters.
- no_grad() вокруг всей базы разорвёт нужные пути.

Мало trainable-параметров ≠ мало всей памяти

Статья	Full fine-tuning	LoRA
Базовые веса	$bW \cdot P$	$bW \cdot P$
Градиенты / states	$bG \cdot P$ / $bO \cdot P$	$bG \cdot p$ / $bO \cdot p$
Активации	Зависят от графа	Зависят от графа

P – параметры базы, $p \ll P$ – параметры adapters.
Веса самих adapters и дополнительные buffers считаем отдельно.

Rank и выбор матриц – разные рычаги

$$d_{in} d_{out} \quad \longrightarrow \quad r(d_{in} + d_{out})$$

Для матрицы 4096 × 4096 и r = 16:

16 777 216 параметров → 131 072 параметра adapters

В 128 раз меньше trainable-параметров этой матрицы.
Target modules определяют, где у модели есть свобода адаптации.

Что известно о качестве LoRA?

- Сравнивать с **full fine-tuning** при настроенном LR и достаточном rank.
- В исследовании LoRA Without Regret важны MLP и выбор target matrices.
- Большой batch в некоторых режимах сильнее ухудшает LoRA.
- Ёмкость адаптера, объём данных и длительность обучения связаны.

Schulman et al.: LoRA Without Regret, 2025

LoRA на inference: merge в базовые веса

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} BA$$

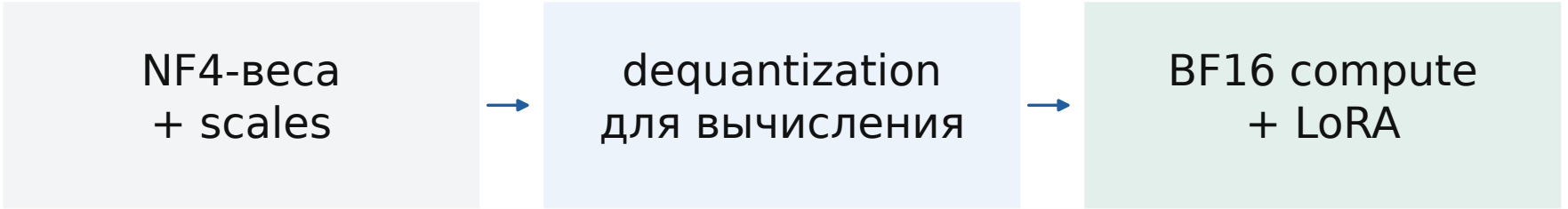
$$W_0 X + \frac{\alpha}{r} BAX = W_{\text{merged}} X$$

- При inference убираем отдельную ветку A/B.
- Остаётся обычная матрица той же формы, без LoRA compute.
- Для переключения задач удобно хранить adapters отдельно.

Eval, dropout выключен. Эквивалентность с учётом floating-point округления.

LoRA: merging weights for inference

QLoRA: сжимаем замороженную базу



$$Y = \mathrm{DeQuant}(W_{\mathrm{NF4}})X + \frac{\alpha}{r}BAX$$

Storage dtype, compute dtype и trainable dtype – разные решения.

База заморожена; адаптеры обучаются в floating point.

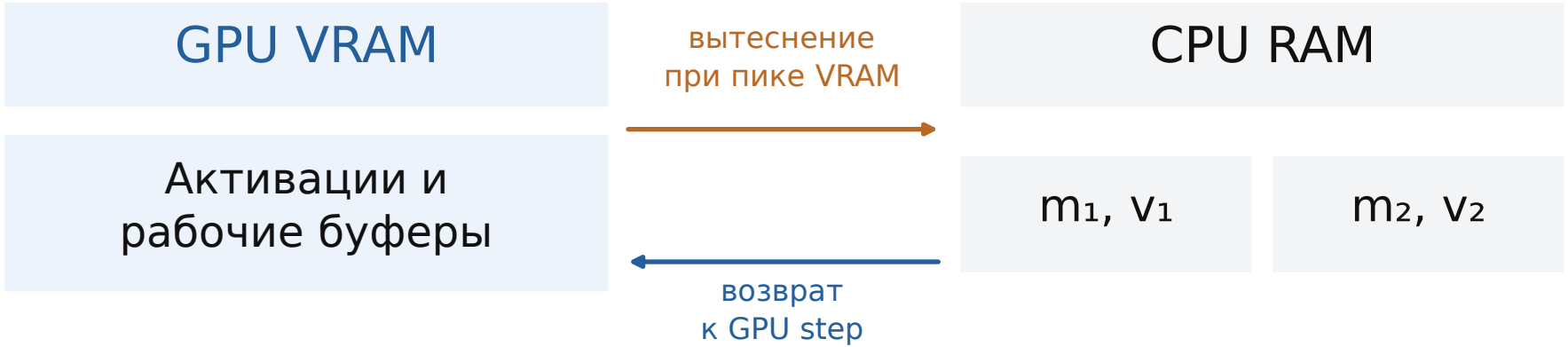
QLoRA, Dettmers et al., 2023

QLoRA: что добавляют остальные компоненты?

- **Double quantization:** квантуем масштабы блоков весов.
- В исходной схеме это экономит около **0,373 бита на параметр**.
- **Paged optimizer:** страницы состояний могут жить в RAM или VRAM.
- Активации и временные буферы всё равно требуют отдельного бюджета.

Paged optimizer: состояния на страницах

Моменты Adam m, v: CUDA Unified Memory, единые виртуальные адреса.



- Во время forward/backward моменты обычно не нужны.
- Перед GPU-обновлением нужные страницы возвращаются.
- **States не исчезли:** изменилось место хранения.

bitsandbytes: paged optimizers QLoRA: memory spikes

Paged optimizer: выигрыш и цена

Что меняем	Что получаем / чем платим
Размещение states	RAM поглощает временный пик VRAM; добавляются миграции
Частые вытеснения	Page faults и трафик по линку могут задерживать GPU
Точность states	32-bit или 8-bit – отдельный выбор, совместимый с paging

PagedAdamW32bit и PagedAdamW8bit: paging ≠ quantization.

QLoRA: merge возможен, но меняет формат или точность

$$W_{\text{float}} = \text{DQ}(W_q) + \frac{\alpha}{r} BA$$

При повторной квантизации в общем случае:

$$\text{DQ}(\text{Q}(W_{\text{float}})) \neq W_{\text{float}}$$

- Сохранить Wfloat: ветка adapters убрана, storage становится больше.
- Вернуть 4 бита: новое округление, качество нужно проверить.

Поддержка merge зависит от quantization backend.

PEFT: merging adapters into quantized weights

Комбинации работают по разным статьям памяти

Комбинация	Что остаётся ограничением
LoRA + checkpointing	База, остаточные активации, compute на пересчёт
QLoRA + checkpointing	Активации, dequantization и временные buffers
PEFT + offloading	Transfer/RAM; states адаптера могут быть уже малы

Сначала выясняем, какая статья памяти теперь доминирует.

А если вообще убрать backward?

LoRA убрала градиенты базы. Но передача градиента по X осталась.

- Forward даёт значение loss; направление изменения можно **оценить**.
- Не строим reverse-mode граф и не сохраняем тензоры для его backward.
- Цена: шум оценки, дополнительные запросы и обновления.

Почему backward часто стоит примерно два forward?

Dense слой: один matmul в forward, два сопоставимых в backward.

forward	backward по X	backward по W
$Y = WX$	$\nabla_X L = W^\top G$	$\nabla_W L = G X^\top$

$$T_B \approx 2T_F, \quad T_{F+B} \approx 3T_F, \quad T_{\mathrm{ZO},\ \mathrm{K}=1} \approx 2T_F$$

3F → 2F: номинально 1,5× на шаге при этой модели стоимости.

Compute-bound эвристика для full training; wall-clock и PEFT могут отличаться.

Два GEMM в backward: оценка FLOPs

ZO: две оценки функции вместо reverse mode

$$\widehat{g} = \frac{L(\theta + \varepsilon u) - L(\theta - \varepsilon u)}{2\varepsilon}\, u$$

$$u \sim \mathcal{N}(0, I), \qquad \widehat{g}_K = \frac{1}{K} \sum_{k=1}^{K} \widehat{g}^{(k)}$$

- Одинаковые данные для «+» и «−» уменьшают лишний шум.
- При конечном ε оцениваем градиент **сглаженной** функции.
- K направлений – 2K оценок функции и работа с параметрами.

MeZO: two-point estimator

За экономию памяти платим качеством оценки

$$L(\theta) = g^\top \theta + c, \qquad \nabla_\theta L = g$$

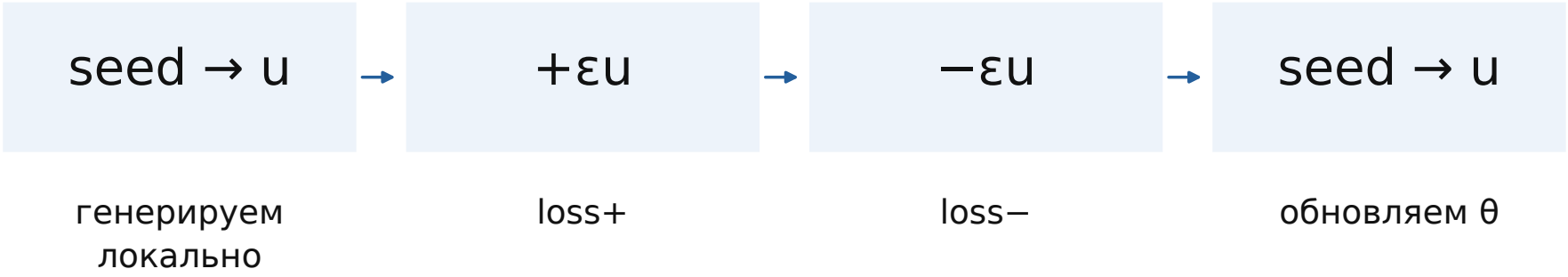
$$\widehat{g} = (g^\top u)u, \qquad \mathbb{E}[\widehat{g}] = g$$

$$\mathbb{E}\left[\|\widehat{g}_K - g\|^2\right] = \frac{d+1}{K}\|g\|^2$$

Больше направлений → меньше дисперсия → больше queries.

В нейросетях важны также effective dimension, структура и предобучение.

MeZO: воспроизводим шум по seed



- Одно направление можно восстановить, не сохраняя полный tensor u.
- Базовый MeZO-SGD не требует reverse-mode activation graph.
- In-place обходы весов и повторная генерация тоже стоят времени.

Forward-only ≠ бесплатно и без состояния

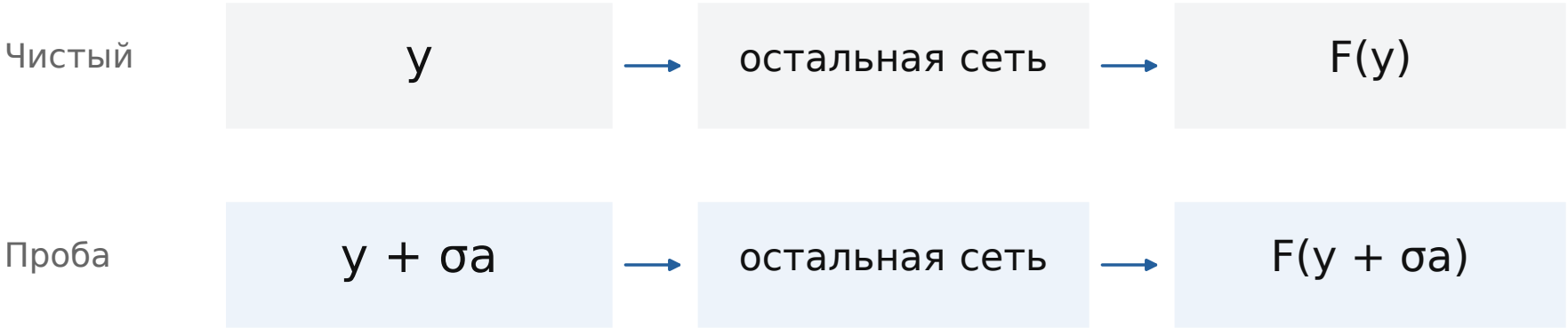
- Два forward могут быть дешевле шага с backward – это надо измерять.
- Но для того же качества может понадобиться больше запросов и обновлений.
- Momentum и Adam снова добавят долгоживущее состояние.
- Параллельная population и кэширование могут увеличить peak memory.

MeZO: fine-tuning, а не обещание pretraining

- Исходная работа исследует адаптацию **предобученных** языковых моделей.
- Предобучение и task prompt помогают работе в локальной области параметров.
- Можно сочетать ZO и LoRA: меняем и пространство, и оценку обновления.
- Сравниваем качество, memory peak и **time-to-target**, а не только step time.

Dust: пробуем изменить выход слоя

$$y = Wx, \qquad F(y) = \mathrm{loss}(\mathrm{suffix}(y))$$



Меняем выход слоя и повторяем оставшийся forward до loss.

Изменение loss измеряет проекцию градиента

$$\Delta_i = F(y + \sigma a_i) - F(y), \qquad a_i \sim \mathcal{N}(0, I)$$

$$\frac{\Delta_i}{\sigma} \approx g^\top a_i, \qquad g = \nabla_y F(y)$$

$\Delta < 0$
проба уменьшила loss

$\Delta > 0$
проба увеличила loss

Из одной пробы получаем одну проекцию градиента.

Почему усреднение «изменение loss × шум» работает?

$$\widehat{g} = \frac{1}{K\sigma} \sum_{i=1}^{K} \Delta_i a_i$$

$$\mathbb{E}[\widehat{g}] \approx \mathbb{E}[aa^\top] g = Ig = g$$

$$\mathbb{E}[a_j^2] = 1, \qquad \mathbb{E}[a_j a_k] = 0 \ (j \neq k)$$

При конечной σ цель оценки – градиент Gaussian-сглаженной Fσ.

Gaussian-smoothed gradient estimator

Из градиента по активации получаем градиент весов

Вход x сохранён из forward. Выходной сигнал ĝ уже оценён.

$$y = Wx \quad \Longrightarrow \quad \widehat{G}_W = \hat{g} x^\top$$

$$\hat{g} \in \mathbb{R}^{d_{out}}, \quad x \in \mathbb{R}^{d_{in}}, \quad \widehat{G}_W \in \mathbb{R}^{d_{out} \times d_{in}}$$

$$\widehat{G}_W = \sum_{t=1}^{T} \hat{g}_t x_t^\top$$

Локальный outer product точный; приближение находится в оценке ĝ.

Dust: estimated output error → weight update

Откуда выигрыш и где остаются приближения?

- Один draw даёт локальные perturbations **во всех позициях** токенов.
- Не нужны отдельные копии всех весов для каждого такого возмущения.
- Локальные оценки Dust добавляют **bias и variance** по сравнению с учебным выводом.
- Clean forward кэшируется, остаются reruns и память промежуточных значений.

Dust, §2.2–2.3: credit и interference

От fine-tuning к pretraining: Noise

Проверка forward-only pretraining: 37,7M параметров, 8×H100.

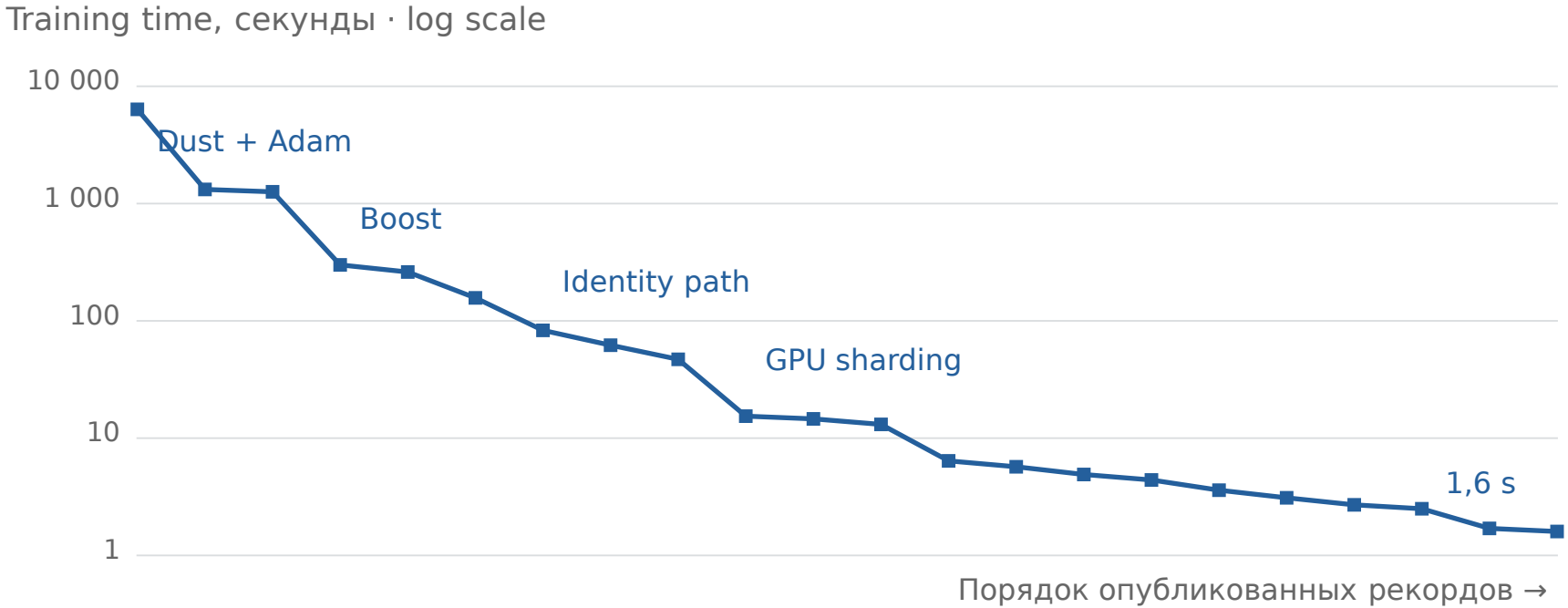
Контракт	Training budget	Speedrun target
Архив до 22.10.2026	≤ 10M токенов, 1 epoch	CE ≤ 4,2
После изменения правил	≤ 30M токенов, 1 epoch	CE ≤ 3,5

Другой порог и budget – другой эксперимент.

Snapshot 26.10.2026 · commit 48e9ea73a7d7

Noise: правила, snapshot 26.10.2026 Архив 10M

От 106 минут до 1,6 секунды



≈ 4000× относительно исходного Dust; target CE ≤ 4,2.

Noise 10M: опубликованные speedrun records

А насколько сильна модель с loss 4,2?

$$\mathrm{PPL} = \exp(\mathrm{CE}_{\mathrm{nats}})$$

Старый порог: CE = 4,2
PPL ≈ 66,7

Новый порог: CE = 3,5
PPL ≈ 33,1

Такой loss легко достигается очень простыми моделями.

Порог 4,2 сам по себе не демонстрирует сильную LM.

Из чего складывался прогресс?

Уровень	Примеры изменений
Estimator / credit	Структурированные probes, identity-path credit
Optimizer / trajectory	ANVIL II, инициализация, averaging, schedule
Системы	Sharding, fused kernels, меньше launches, overlap

Оптимизатор, оценка сигнала и реализация менялись вместе.

Так что именно мы хотим ускорить?

У рекорда есть порог. У нашей задачи есть нужное качество.

$$T_{\text{target}} = \sum_{k=1}^{N_{\text{target}}} T_k$$

Стоимость одного шага	×	Сколько шагов нужно до цели

Слева – система и kernels. Справа – оптимизация и данные.

Произведение – при примерно постоянной стоимости шага.

Как собрать решение под одну GPU?

Baseline: известные AMP и эффективные kernels, затем измерение.

Нужен другой batch	→	Microbatch + accumulation
Доминируют активации	→	Checkpointing / selective / offload
Доминирует состояние	→	PEFT / optimizer / quantization
Готовы менять алгоритм	→	ZO с проверкой time-to-quality

После каждого изменения узкое место может переместиться.

За что платим, когда экономим память?

Приём	Основная цена
Accumulation	Последовательные microsteps и возможная потеря throughput
Checkpointing	Повторные вычисления и управление RNG
Offloading	Передачи, RAM, зависимости и staging buffers
LoRA / QLoRA	Ограничение обновления / ошибка и стоимость quantization
ZO	Шум оценки, queries и возможные дополнительные шаги